当代人工智能实验课　实验二 搜索算法

姓名：张凌琨　　学号：10241900443

实验目标与研究问题

实现 A*、Minimax 和 Alpha-Beta 后，我用四个正式样例核对答案。接着比较访问量和总时间，检查同一改动在不同地图和树上是否仍有效。遇到异常结果，我先固定输入和其他设置，再改变一个条件寻找原因。

输入输出与实验方法

四题输入输出约定

坐标均为从 0 开始的(row,col)。题一、二使用三段文本：起点一行、终点一行、随后为可跨行二维列表，字符须加引号；均支持--input 文件与--stdin。

题目	主要输入内容	正式输出
一	S/E/./X 字符矩阵	含起终点的路径；无解[]
二	非负整数及 S/E/X 矩阵	最小进入费；E 免费，无解 −1
三	JSON 树，内部 children、叶 value；可包 tree_structure	MAX 根值，逐层交替 MAX/MIN
四	JSON 的 grid 与 treasures 数组；元素含 id、position、value	完整 6×6 矩阵及 H2/H4 首步、U

表 1 四题输入结构与输出格式

题四 A/B 由地图标记或 starts 指定，H 由--depth 选择，默认同时跑 2/4；不读取 JSON 的 H 字段。四题省略输入参数时运行内置样例。题一、二也提供返回路径和统计的详细接口。

算法伪代码

g 为最小已知到达费，h 为剩余费用下界，f=g+h。OPEN 以 f 为主键，同 f 默认 FIFO；low_h 仅在同 f 时优先小 h。算法 1 省略次关键字和序号，保留入堆时的 g，以跳过过期记录。

```
算法 1 A*(S,E)
g[S]←0; g[others]←∞; parent←{}
OPEN←{(h(S),S,0)}
while OPEN≠∅:
  (f,u,gu)←pop_min(OPEN)
  if gu≠g[u]: continue
  if u=E:
    return Result(backtrack(parent,E),gu)
  for legal v: if gu+c(u,v)<g[v]:
    g[v]←gu+c(u,v); parent[v]←u
    push(OPEN,(g[v]+h(v),v,g[v]))
return no_path
```

Result 含路径和代价，正式输出按表选取；no_path 在题一/二映射为[]/−1，题四不可达距离为∞。有效目标出队才结束，严格松弛防止零权环。c 在题一为 1，在题二为进入费且 E=0；题四命名格费为 1。

```
算法 2 AB(s,d,α,β)
if Stop(s,d): return Eval(s)
b←−∞(MAX) or +∞(MIN)
for a in ordered_children(s):
  v←AB(a,d-1,α,β)
  b←max(b,v)(MAX) or min(b,v)(MIN)
  α←max(α,b)(MAX); β←min(β,b)(MIN)
  if α≥β: break
return b
```

α 是 MAX 当前可保证的下界，β 是 MIN 当前可保证的上界；α ≥ β 时，后续分支已无法改善祖先的选择。题三 Stop 仅为真实叶，Eval 为叶值，d 不限制搜索；题四在 d=0、宝藏取完或无合法动作时返回当前 U，每次孩子递归把 d 减 1。根窗口为(−∞,+∞)，内部 fail-soft 回值可能是界。Minimax 采用相同终止和评价，递推 V=max/min 孩子 V，取消窗口更新及剪枝即可。题四另记录改善 b 的首步和主变例。

随机生成与控制实验设计

手工反例用于定位机制，固定种子随机图/树用于检验适用范围；无解与未提速输入均保留。下表分别统计输入记录、参数配置与计时重复，避免重复计算样本量。

研究组	生成与固定条件	对照目的
题一	3 尺寸×5 障碍率×20 种子=300 记录；p=0 为重复基线	同图改 h/排序；有解无解分层
题二	200 个不同地图一端点组合，含随机费用与零权；桶另设 12 图×7 倍率	同图免费 E；下界/排序/容器消融
题三	Scout 含 18 棵新随机树及 4 组镜像结构；共享度另设组	同树同序；计入全部探测重搜
题四	30 个固定种子跨图输入；缓存另用 4/6/8 宝藏各 6 新输入	同深度 MM/AB、顺序和缓存控制

表 2 各研究组的生成规则与控制变量

Windows 11、64 位 Python 3.12.14，代码仅用标准库。新实验先固定输入和比较方案，再预热并串行执行 7 个随机顺序块；题二/三/四每块分别 2/3/1 次，取块均值的中位数与 IQR。既有地标、缓存确认用 15 块。寻路计入准备与回溯，博弈计入排序、查表及重搜；题四另测距离矩阵与博弈的合计时间。轨迹关闭，不跨轮比较绝对毫秒。

独立 BFS/Dijkstra 核验路径与费用，完整枚举核验博弈值和收益账本。A*扩展不含目标，博弈访问含根，缓存直接返回另计；237 项测试通过。

题目一 迷宫寻路

最短路径怎样得到

题一以当前位置(r,c)为状态，每次上下左右移一格，边代价均为 1。曼哈顿距离 h(n)=|r−rE|+|c−cE| 是剩余步数下界：一步最多使行列差的绝对值之和减少 1，障碍只可能迫使绕行，不能缩短这个必要距离。相邻格的 h 最多相差 1，所以还满足 h(n)≤1+h(n′)，即一致性。

正式样例返回路径为：

```
(0,0) → (0,1) → (0,2) → (1,2) →
(1,3) → (1,4) → (2,4) → (3,4)
```

路径包含 8 个坐标，对应 7 次移动。起终点曼哈顿距离也是 7，因此该路径达到下界；独立 BFS 同样得到 7。沿这条路径，g 从 0 增到 7，h 从 7 降到 0，f 始终为 7。

下面在相同输入上比较 h=0、曼哈顿/FIFO、曼哈顿/low_h。h=0 使 A*退化为一致代价搜索；本题单位费用下与 BFS 按相同距离层次搜索，但仍使用堆，运行开销不必相同。

场景	最短步数	h=0/FIFO 扩展	曼哈顿/FIFO 扩展	曼哈顿/low_h 扩展
PDF 样例	7	15	13	7
开阔 40×40	78	1599	1599	78
强制绕路	64	588	527	318

表 3 相同地图下的启发与同分策略

长墙使曼哈顿 20 的查询绕行至 64 步，FIFO 仍扩展 527 格。另测固定无解图，三种设置均遍历起点所在的 105 格连通区域，启发函数未减少这部分扩展。

为什么准确启发仍会搜索很多

开阔 40×40 图上，曼哈顿已等于真实剩余距离，FIFO 仍和 h=0 一样扩展 1599 格。我固定地图、端点和 h，只把同 f 排序改成 low_h。每格最优 g=r+c，h=78−r−c，f 恒为 78，主键无法区分候选。FIFO 沿波前推进，low_h 优先取 h 更小的格子，扩展降至 78 格。

同轮时间中位数由 4.52516 降至 0.37323ms。沿一支深入时，其他候选积在堆中，堆峰值从 40 增至 77。这组对照中，访问量下降伴随更大的待选队列。正式样例仅差 2 次扩展，时间差不足 0.001ms。

地标数量与总运行时间

静态地图上借鉴 ALT，用 BFS 预计算地标 L 到各格的距离，以 max(曼哈顿, |d(L,n)−d(L,E)|)作下界，三角不等式保证可采纳性。选点不使用查询答案。每次从独立建表计至 40 查询返回路径，扫描 k=0、1、2、3、4、8 后，固定 k=1 与两侧 0、2，再采 15 个计时块。

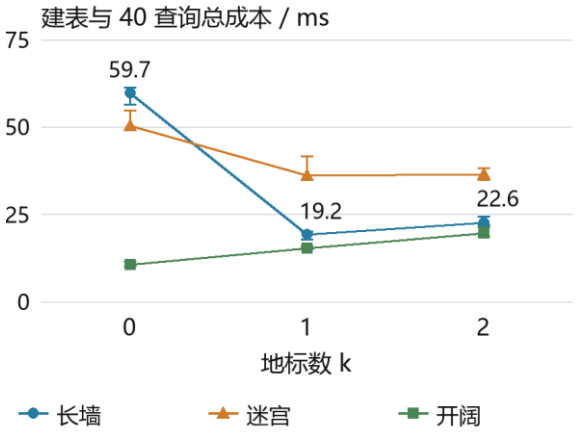


图 1 建表与 40 查询总成本。点为 15 块中位数，竖线为四分位区间[Q1,Q3]

长墙组 k=0/1/2 总时间为 59.703/19.162/22.629ms。增加第一个地标后累计扩展 18483→2068，第二个地标未再减少扩展；k=1 对两侧均 15/15 块更快。T1−T2 配对差中位−3.235ms，四分位区间[−5.520,−2.657]ms：继续增加地标只增加建表、查表开销，未带来相应的搜索节省。

迷宫 k=1/2 为 36.131/36.552ms，配对差+0.084ms 且区间跨 0，是近似平台；开阔图 k=0/1/2 为 10.587/15.316/19.570ms，扩展都为 1314，不建表最好。低点仅对固定长墙任务成立；15 块复用原 3 图，不是新增 15 图。

走廊压缩是否与地标互补

借鉴宏边保距思想，将度 2 内部格收缩为等长边，保存原路径；查询端点动态切回，并保留纯环锚点、平行边。任意简单最短路可双向等长转换；宏边长度≥端点曼哈顿，故 h 仍一致。建图 O(RC+V+E)，查询后须付出逐格输出成本。

新增 18 张不同地图，41/61/81 三尺度，1/4/16/64 查询前缀，每配置 7 块，比较 10 种实现。64 查询时，压缩/原版 low_h 总耗时比的跨图中位：迷宫 0.155（6/6 图更快）、长廊 0.067（3/3）、随机障碍 1.119（1/6）、开阔 1.819（0/3）。计时含建图、端点接入和路径还原。

预选 61×61 迷宫中，同一实现关闭/开启压缩为 191.964/29.014ms，说明时间下降不只是省去了重复地图校验；压缩加地标为 25.224ms，优于原地标 117.906ms。迷宫在所测 4 或 16 查询首次回本，开阔至 64 次仍未回本。压缩节点与原格点代表的工作量不同，节点数不能直接等量比较。1152 次查询均经独立 BFS 核验，返回路径保持最短。

题目二 加权地形

逐格收费与免费终点

题二最小化费用而非步数。边 u→v 的代价是进入 v 的权值，起点初始 g=0；按题目规定，进入 E 的最后一步收费 0。正式样例的一条最优路径及费用如下。

次序	坐标	进入费用	累计费用
0	(0,0)	0	0
1	(1,0)	1	1
2	(2,0)	1	2
3	(3,0)	1	3
4	(3,1)	1	4
5	(3,2)	1	5
6	(4,2)	1	6
7	(4,3)	1	7
8	(4,4)	0	7

表 4 题二样例路径的逐格进入费用

共 8 步，中间 7 次进入普通格各收费 1，最后免费，总费 7。低费绕行也可能比步数较少的高费路线更好，因此不能用 BFS 的最少步数直接代替本题目标。实现保存不可变权重快照，删除返回起点的边；非负费用下，任何回到起点的路径都含可删除的非负环，这不会损失最优解。

曼哈顿距离是否仍可采纳

原始曼哈顿距离在本题中不可采纳。若所有进入费都至少 1，M=Manhattan 可作下界；但本题进入 E 免费，邻 E 状态的真实剩余费用为 0 而 M=1，已经高估。

令 m 为普通可通行格最低进入费，取下界：

$$h_c(n) = m \times \max(M(n,E) - 1, 0)$$

任意路径至少走 M 步，其中最后一次进入免费，其余至少 M−1 次收费不低于 m。相邻非终点边上 h_c 至多下降 m，不超过进入费用；进入 E 前 h_c 已为 0，因此免费边也满足一致性。如果普通格含 0 权，m=0，h_c 安全退化为零启发。

为检验高估会不会改变答案，我用上排 S、1、E 和下排 0、0、0 构造 2×3 网格。直达收费 1，绕下排收费 0。原始曼哈顿高估绕行的余费并提前返回 1，校正版得到 0。这个零权反例补足了 PDF 正权样例无法检验的情形。

普通格进入费用均至少 1 时，普通 M 虽不满足可采纳性，仍可能在本模型中返回正确费用：它与 max(M−1,0) 在非终点上只差常数 1，目标前驱的有效弹出已达到最优代价。这是一项依赖费用条件的性质；加入其他 0 权格后，常数差不再保护搜索。实际零权随机拓展的 29 个有解输入中，raw_M 有 11 个返回非最优费用，安全配置全部正确。

把地形结构加入下界

全局 m 忽略了高费用出现在何处。为保留部分位置结构，令 a_r、b_c 分别为每行和每列的最小进入费；将从当前位置走向目标行所需进入行的最小费相加，再加走向目标列所需进入列的最小费，得到 h_{rc}。每一步最多只减少其中一个方向的项，减少量不高于实际进入格费用，因此 h_{rc} 一致；与 h_c 取最大值仍安全。

行列下界还可以收紧：它重复使用了一次免费终点。目标行和目标列都因含 E 而最小费为 0；若当前位置行列都未对齐，路径必须先进入目标行或目标列，再到达二者交点 E。这两次进入不可能都免费，至少一次目的格不是 E，费用至少 m。因此定义收紧版本：

$$B(n) = h_{rc}(n) + m \times I[r \neq r_E \text{ 且 } c \neq c_E]$$

$$h_t(n) = \max(h_c(n), B(n))$$

指示项从 1 降为 0 的那一步只是刚对齐一轴，目的格尚非 E，其至少 m 的费用覆盖了新增下降量；进入 E 的最后一步前后指示项均为 0。其他相邻情况沿用行列下界的一致性，故 h_t 仍可采纳且一致。行列数组在搜索前建立，单次估计为常数次查表。

我用最优费用为 83 的 31×31 高费行列带检验收紧效果。固定 low_h 时，校正、行列和收紧下界的起点估计为 59、82、83，扩展分别为 850、811、60。我再换回 FIFO，并按 f 相对最优费用 C*的位置拆分节点数，检查这一下降依赖哪些条件。

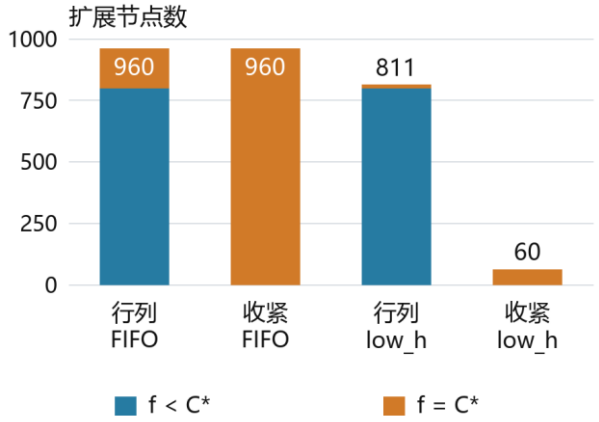


图 2 启发强度与同分排序共同决定最优 f 平台上的扩展

相同 FIFO 下，row_column 有 800 个 f<C* 状态和 160 个 f=C* 状态，总计 960；tight 没有 f<C* 扩展，却仍有 960 个 f=C* 扩展，总数不变。换成 low_h 后，前者为 800+11=811，后者为 0+60=60。更强下界消除了最优边界之前的扩展，也可能把很多状态集中到边界；是否继续访问旁支由同分次序决定。

收益来自下界与排序的共同作用。普通 0 权使 m=0，tight 附加项消失；均匀正权图上几个安全下界也可能给出相同节点数。基础同轮高费带 corrected/tight 为 4.443/0.823ms，零权高费带虽少扩展却可能更慢。

整数队列与费用单位归一化

一致整数 h 使 d=c(v)+h(v)−h(u)非负。我用 D=max d 构造 D+1 个循环桶，桶内沿用同分规则，超过 32768 槽便回退到堆。归一化以正进入费的 gcd 换算桶下标，费用账本仍用原单位。

我在 12 张图上把普通费乘 q=1、2、4、8、16、64、256，E 保持免费。堆与桶的路径、有效出队次序一致，便可单独观察费用范围的影响。计时含 h 表、gcd 和 D 扫描，并对照同模块堆及原 tight+low_h。

31×31 零权随机图取 q=256 时，槽数 1793→8，空扫 13568→53。两种桶均扩展 836 次，耗时 14.046→8.907ms，原版为 6.090ms。三张零权图的归一化/原桶耗时比中位在 q=256 时为 0.657，在 q=1 时为 1.031。168 组配对比较均未快过原版，正式程序沿用原堆。

题目三 极大极小博弈树的最佳得分

样例回传与实际剪枝

正式 JSON 中的四个 MAX 子树叶对为(3,5)、(6,9)、(6,9)、(1,2)，完整子树值为 5、9、9、2。两个 MIN 分别回传 5 和 2，根 MAX 得到 5。题目示意图较简略，输入实现以完整 JSON 为准；Python 示例中的共享对象也保持同一展开博弈值。

左 MIN 的第一个孩子已返回 5，因此其后 MAX 的 β=5。该 MAX 读到 6 后已有 α=6≥β=5，另一个叶 9 可以跳过，返回 6。这里 6 只说明该子树值至少为 6，完整真值仍为 9；左 MIN 已经有更好的 5，所以不需要知道更大的确切值。

Minimax 共访问 15 次、读取 8 叶，左到右 Alpha-Beta 访问 14 次、读取 7 叶。样例只省一个叶，策略准备与搜索时间中位约 0.0389/0.0393ms，几乎相当。右 MIN 在最后一个孩子后也可能发生窗口交叉，但没有兄弟可省，因此“界交叉次数”与“有效剪枝次数”分开记录。

顺序怎样影响搜索与准备成本

我固定同一棵树和叶值，只改变子节点顺序，观察剪枝发生的位置和省下的访问量。真值优先序和逆序先完整求值，用来查看顺序可能带来的差距，准备时间也计入总成本。下表用分支数 4、深度 6、种子 20261030 的树。各方法根值一致。

方法	搜索访问	搜索叶	准备真值叶读取	策略准备与搜索中位 ms
Minimax 左序	5461	4096	0	11.626
Alpha-Beta 左序	1817	1181	0	4.989
Alpha-Beta 右序	700	451	0	1.843
Alpha-Beta 固定随机	1422	915	0	8.539
Alpha-Beta 真值优先	268	127	4096	7.482
Alpha-Beta 真值逆序	4720	3462	4096	22.915

表 5 同一棵树的访问顺序与策略成本

真值优先把搜索叶数从左序的 1181 降到 127，事前却要读 4096 叶。计入这部分开销，真值优先总耗时为 7.482ms，左序为 4.989ms。右序在这棵树上较早形成界。

这个对照让我把排序获得的剪枝和产生顺序的代价一起计量。

三分支六层、叶值全为 7 时，$\alpha \geqslant \beta$ 访问 125 次、读 53 叶；严格 $\alpha > \beta$ 则为 1093/729，与完整 Minimax 相同。相等已经足以排除后续分支，要求严格超过界会错失节省；两者根值仍正确。

缓存必须区分精确值与界

共享子树在不同历史中可能被反复求解，适合尝试置换表。缓存键包含节点对象及当前玩家，每次调用新建表，保存 exact、lower、upper 三类结果。精确项可直接复用；下界达到 β 或上界不高于 α 时可直接截断，其余命中继续按原窗口搜索。实现没有用命中界去收紧窗口，界类型决定能否直接回传。

一个小反例足以说明必要性。令 X=MAX(6,9)，根为 MAX(MIN(5,X), MIN(X,8))。左分支在 $\beta=5$ 下搜索 X，读 6 后截断，返回下界 6。若误将 6 缓存为精确值，右分支会得到 MIN(6,8)=6，根错算 6。实际上 X=9，右分支为 8，根应为 8。安全缓存保留界类型，第二次窗口不能被该下界关闭时继续搜索，最终得到 8。

性能实验保持有序展开的叶值与结构语义相同，只改变重复引用程度。分别检验强共享、部分共享与纯树。

强共享代表输入中，原版访问 958 次、读 530 叶，界感知缓存访问 167 次、读 10 叶，时间中位由 4.159 降至 1.524ms；无共享纯树则两者都访问 958 次，时间由 4.332 增至 9.615ms。查表和维护成本只有在省下足够多的重复搜索时才能摊薄。

固定策略后，三个新种子的强共享配对差中位为 −1.209、−1.265、−1.492ms，四分位区间均低于 0，更快块为 14/15、15/15、15/15。纯树全部块更慢。结论限于同次求解内稳定的重复对象；结构相同但对象不同不自动合并，容量增加主要呈饱和平台。

Scout 探测省下的求值与重搜代价

Scout 先精确求第一个子节点，其余仅用布尔 TEST 判断能否严格改善候选，能改善才重新精确求值。TEST 只回答能否严格改善候选，直接比较整数或浮点数，无需设置 ε。输入为完整显式树，内部节点没有截断评价函数，因此未加入迭代加深。所有 TEST 与 EVAL 重复访问都计入，与同模块 AB 及原版按相同顺序比较。

新增 18 棵随机树、4 组成对结构和 4 个特殊输入，共 30 份有序记录。左序首候选最佳的 4 组，Scout/同模块 AB 时间比中位 0.733，总读叶比 1；镜像成后置逐次改善后，时间比 1.980、读叶比 2.900。18 随机树左序只有 8/18 配对更快，读叶比 1.625，不能普遍替换 AB。与原版的时间差还受实现及统计开销影响，因此主要用同模块 AB 判断 Scout 策略的作用。

题目四 竞争环境下的宝藏分配（综合题）

距离矩阵与博弈状态

题四将寻路结果作为博弈每步收益的组成部分。A、B、宝藏所在格进入费均为 1，不能沿用题二的终点免费规定。四方向 A* 使用 $h=m\times\text{Manhattan}$，逐一运行 30 个有序不同位置对，另令对角为 0。样例端点费用相同，矩阵自然对称；实现仍实际计算全部有序对。

来源/目标	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

表 6 六个命名位置之间的最小移动费用

使用曼哈顿时，样例完整矩阵累计扩展 329 次；零启发为 650 次，全部费用一致。两者构造时间中位分别 2.569/3.863ms。一般数值端点的进入费用可能不同，最短费用不能预设对称，测试也覆盖了这一情形。

博弈状态记录双方位置、剩余集合、累计收益和当前玩家：

```
(pos_A, pos_B, remaining_mask,
 score_A, score_B, turn)
```

递归另传剩余深度。选择 Ti 后，只更新行动者位置，清除剩余位，累计 Gain=Vi−d(pos,Ti)，再切换玩家。经过其他宝藏不自动获得，已取格仍可通行，另一玩家也不阻挡路径。即使 Gain 为负也必须选择，因为题目没有 pass 动作。

H 每增加 1 代表一个玩家选取一次，因此 H=2 是 A→B，H=4 是 A→B→A→B。叶评价为当前 $U=\text{score}_A-\text{score}_B$。四宝藏完整 Minimax 在 H4 有 65 个访问节点和 24 叶，叶对应 4!种选择排列；Alpha-Beta 保持相同目标，只省去不影响祖先选择的分支。

累计收益是必要状态信息。例如两条合法历史 T1→T2→T3→T4 与 T2→T1→T3→T4，结束时双方位置、剩余集合与轮到的玩家相同，U 却为 1 和 7。若缓存绝对 U，不能只按位置和剩余集合合并。题三的对象缓存也因此不能直接照搬成综合题状态缓存。

H2 与 H4 为什么首步不同

两算法同深度根值一致。H2 的 Minimax 访问 17 次，Alpha-Beta 标签/逆/Gain 序分别 13/16/11 次；H4 为 65 次和 47/46/37 次。Gain 序提前收紧界，却要支付排序成本；默认仍按标签访问。

为查清深度为何改变首步，我依次固定 A 的每个选择，再求该深度的最优续走。表中的每个分支另用完整求值核对，包含原搜索中被剪掉的分支。

H	T1	T2	T3	T4	最优首步	最优 U
1	−1	2	−1	−1	T2	2
2	−4	−1	−2	−4	T2	−1
3	−4	2	−1	−4	T2	2
4	−5	1	3	−7	T3	3

表 7 不同深度下四个首步的精确分支值

H2 先取 T2，A 即时收益 6−4=2，B 随后取 T3，收益 7−4=3，U 为−1。H4 的一条最优延续为 T3→T1→T2→T4，收益按各玩家更新后的位置累计：

回合	玩家	移动	价值	路费	当次 Gain	累计 U
1	A	A→T3	7	8	−1	−1
2	B	B→T1	9	8	1	−2
3	A	T3→T2	6	6	0	−2
4	B	T1→T4	7	12	−5	3

表 8 H4 一条最优延续的逐回合收益

沿这条延续，A 绝对收益为−1，B 为−4，收益差仍为 3。另一条并列最优延续可得到 A=2、B=−1，U 同为 3，因此双方绝对分数并不唯一；目标是相对收益差。最后一步即使亏 5 仍必须执行，也是完整博弈与只看即时 Gain 不同的原因。

H2 只计双方第一次行动，未计第二次移动及剩余宝藏竞争；H4 覆盖全部四回合，因此首取 T3 虽即时亏 1，最终相对收益更好。H2 的−1 与 H4 的 3 处在不同截断时点，不能直接把差 4 称为首步策略提高 4。统一用 H4 完整续走评价：浅层首选 T2 的分支值为 1，最优 T3 为 3，固定该首步造成的完整后悔值为 2。

宝藏价值的两个决策边界

样例 v=7 时两种深度选了不同首步。我固定地图及其他价值，只把 T3 价值 v 从 0 扫到 14。H2 首取 T2、T3 的分支值分别为 2−max(1,v−4,−3)和 v−9，在 7.5 相交。H4 四分支为 2−v 、8−v 、v−4 、−v ，最优 U=max(8−v,v−4)，T2/T3 在 v=6 相交。正式 v=7 正好位于两处交点之间。

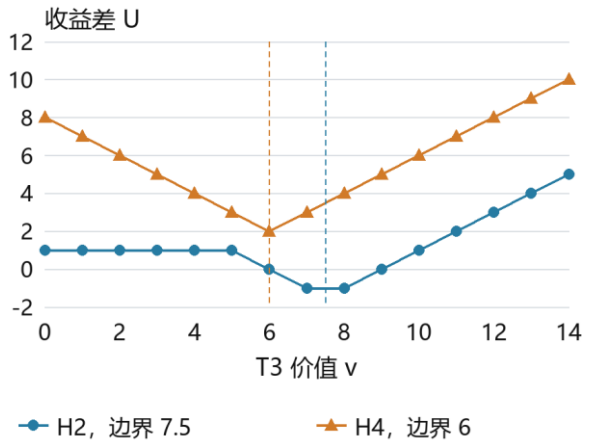


图 3 只改变 T3 价值时的最优收益差。虚线标出 T2 与 T3 的切换点

整数扫描给出切换区间。我在解析交点做定点核验，两种算法各试三种访问顺序。H4 在 v=6 时 T2、T3 同为 2，H2 在 v=7.5 时同为−1.5。标签序和 Gain 序返回 T2，逆序返回 T3，各自的最优值一致。H4 曲线先降后升，正对应两条首步分支在交点两侧分别占优。

跨地图检验排序与浅层决策

另外生成开放加权、单缺口墙和随机障碍各 10 图，只按连通与命名点互异等结构接收，不按性能筛选。全部距离对照 Dijkstra，H1 至 H4、两算法、三顺序对照完整枚举。

H4 标签序 AB 在 30 图均少于 Minimax 访问，时间却仅 24 图更短。Gain 相对标签序 22 图少访问、3 图相同、5 图更多，时间 19 图快、11 图慢。以同图 H4 完整续走评价浅层首步，H1/H2/H3/H4 命中最优为 19/19/23/30 图；后悔值中位虽均 0，H2 最大损失达 32。

加深未必逐级改善。例如 open_weighted_s202611104 的 H2/H3/H4 完整后悔值为 1/6/0；H3 截于 A 第二次行动，未计 B 最后取宝，当前 U 更乐观。全部 30 图有 2 图出现 H2 到 H3 变差。

收益增量缓存何时有效

绝对收益不同的历史不能直接合并，但未来规则不依赖已有分数。令 x=(posA,posB,mask,turn)，可归纳得 F_h(state)=U+V_h(x)，V_0=0；A 取 max(Gain+V_child)，B 取 min(−Gain+V_child)。状态仍记录双方累计收益，缓存只存增量，键保留精确剩余深度。缓存区分精确值和上下界；取出相对界后先加当前 U，再与 α / β 比较，不能直接截断则继续原窗口。

六宝藏反例中，两条历史到相同后续局面，当前 U 为 7/3，最终值为 10/6；未来增量均 3。去掉分数却缓存最终 10 会算错，缓存增量 3 才安全。仅缓存非终止状态：四宝藏完整展开 41 条非终止历史对应 41 个结构状态，没有复用；六/八宝藏为 1237/457、69281/3537。结构计数只说明可能复用，实际访问仍受 AB 剪枝影响。

新增 18 个不同地图/端点/价值输入，每个规模 6 图，比较原版及同模块的无缓存、完整分数键、增量键，两算法与两顺序。缓存限整数，单次调用清表。完整任务实际计算 30/56/90 次有序 A*距离。八宝藏标签/Gain 序的增量 AB/原 AB 总耗时比中位为 0.557/0.750，两顺序均 6/6 图更快；与同模块无缓存对照的标签序比 0.671，也支持缓存本身获益。四宝藏零命中，六宝藏未稳定回本；各规模地图不同，不能由此断言精确宝藏数阈值。

既有延迟查询在 31 图均减少查询，时间相对原版却仅 3 图更快；完整矩阵仍须 30 次 A*。因此新增缓存同时报告纯博弈与完整任务，不能把内部工作减少直接等同总成本下降。

结论与适用范围

两次复验改变了我对参数的判断。在 101 阶带状图的初轮对照中，原版中位耗时约是堆清理 ρ=1.5 的 1.11 倍。我固定 ρ=1.5，在同一输入上重采 15 个计时块。堆峰值由 1192 降到 386，但只有 2 块比原版快，配对耗时差中位数为+3.769ms。这个结果保留了队列变小的证据，默认求解仍采用原版。

采样排序在探索的 6 棵树上，q=8 附近的聚合访问量较低。我预先固定候选，换 30 棵同分布新树复验。q=8 在 25 棵树上访问更少的节点。计入采样准备后，它在 29 棵树上慢于不采样。前一组复验检查同一输入上的计时稳定性，后一组检查换输入后结果能否重复。以后选择默认方案时，我会分别核对正确性、整次调用耗时和跨输入的一致性。